Building a Charity & Fundraiser Platform with MongoDB

by TEJASVI VINOTHKUMAR (241001288)

Made with GAMMA

Experiment 1: Basic Collection Operations

Creating Collections

The simplest way to start managing charity data is by creating collections without constraints.

```
db.createCollection("temp_campaigns")
```

This command creates a new collection called temp_campaigns to store campaign data temporarily.

Viewing Collections

Verify your collection exists by listing all collections in the database.

```
show collections
```

This displays all collections in the current database, helping you confirm creation.

Made with GAMMA

Experiment 1: Dropping Collections

1

Target Collection

Identify the collection you want to remove from your database

2

Drop Command

Execute the drop method to permanently delete the collection

3

Verification

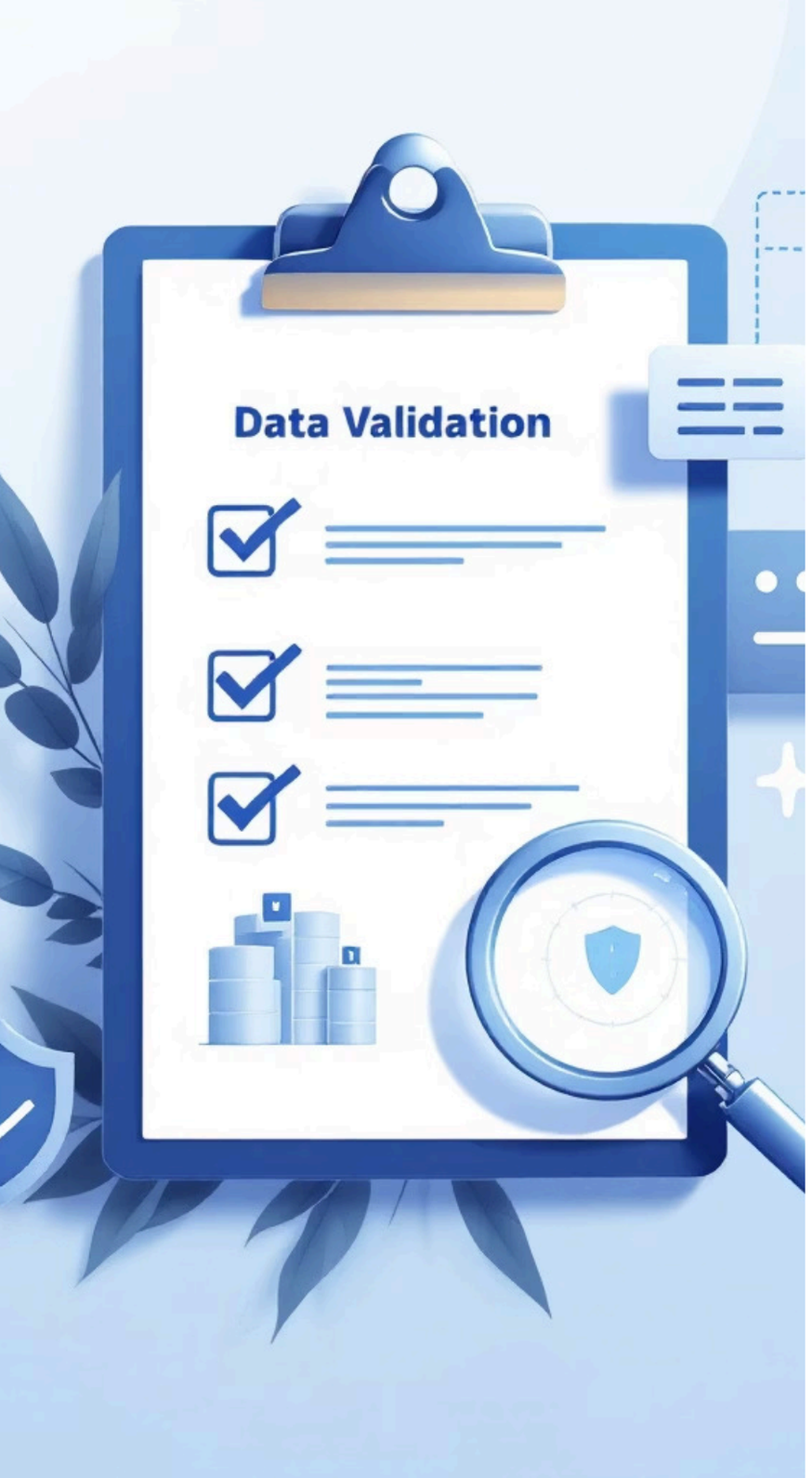
Run show collections again to confirm the collection is gone

Important Note

db.temp_campaigns.drop() - This command permanently deletes the collection and all its documents. Use with caution in production environments.

Result: The collection was successfully created and dropped, demonstrating basic collection lifecycle management.

Made with GAMMA



Experiment 2: Schema Validation Setup

Enforcing Required Fields

Build data integrity into your collections by requiring specific fields and data types.

```
db.createCollection("campaigns", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["campaignTitle", "campaignId", "organizationName"],
 properties: {
 campaignTitle: { bsonType: "string" },
 campaignId: { bsonType: "string" },
 organizationName: { bsonType: "string" },
 goalAmount: { bsonType: ["double", "null"] },
 isActive: { bsonType: "bool" }
 }
 }
 }
})
```

Made with GAMMA

Experiment 2: Valid Document Insert

✅ Valid Document

All required fields are present with correct data types:

- campaignTitle (string)
- campaignId (string)
- organizationName (string)
- goalAmount (double)
- isActive (boolean)

```
db.campaigns.insertOne({
  campaignTitle: "Clean Water for All",
  campaignId: "CMP2025001",
  organizationName: "AquaHope NGO",
  goalAmount: 500000.5,
  isActive: true
})
```

Result: Document successfully inserted into the campaigns collection. The validator approved this document because it contains all required fields with matching data types.

Made with GAMMA

Experiment 2: Invalid Document Rejection

❌ Missing Required Field

The campaignId field is missing from this document, violating the required fields constraint

```
db.campaigns.insertOne({
  campaignTitle: "Feed the Children",
  organizationName: "HopeFoundation"
})
```

What Happens?

MongoDB rejects the insert operation and returns an error message. The document never gets stored, protecting data integrity.

Result: The collection enforces required fields. Invalid documents are automatically rejected, preventing incomplete campaign data from entering the system.

Made with GAMMA

Experiment 3: Advanced Validation Rules

String Length

Set minimum and maximum character limits for donor names

Enum Constraints

Restrict donation categories to predefined options only

Range Checks

Enforce minimum donation amounts and unit counts

Pattern Matching

Validate transaction IDs using regular expressions

Made with GAMMA

Experiment 3: Validation in Action

✅ Valid Insert

```
db.donations.insertOne({
  donorName: "Priya Sharma",
  donationCategory: "Medical Aid",
  amount: 7500.0,
  unitsReceived: NumberInt(10),
  transactionId: "TXN123456AB"
})
```

All constraints pass:

- Name length: 3-100 characters ✓
- Valid category ✓
- Amount > 50 ✓
- Units ≥ 1 ✓
- Transaction ID pattern ✓

❌ Invalid Insert

```
db.donations.insertOne({
  donorName: "Arjun Mehta",
  donationCategory: "Sports",
  amount: 1000.1,
  unitsReceived: NumberInt(5)
})
```

Validation fails because:

- "Sports" not in enum list ×
- transactionId missing (required) ×

Result: Advanced validation ensures correct donor data and prevents invalid entries from corrupting the database.

Made with GAMMA

Experiment 4: Modifying Existing Collections

The Challenge

What if your collection already exists but needs validation? MongoDB provides `collMod` command to add rules without recreating the collection.

This is perfect for legacy systems where dropping and recreating collections isn't feasible.

Made with GAMMA

Experiment 4: Implementation Steps

1

Create Collection

Initialize without validation

```
db.createCollection("volunteers")
```

2

Run collMod

Add validator rules

```
db.runCommand({ collMod:
"volunteers", validator: { ... } })
```

3

Set Level

Choose strict or moderate validation

Pro Tip: Use `validationLevel: "moderate"` to allow updates to existing documents while enforcing rules on new inserts

Made with GAMMA

Experiment 4: Complete Query

```
db.runCommand({
 collMod: "volunteers",
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["volunteerId", "fullName", "role", "hoursCommitted"],
 properties: {
 volunteerId: {
 bsonType: "string",
 pattern: "^VOL[0-9]{5}$"
 },
 fullName: {
 bsonType: "string",
 minLength: 3
 },
 role: {
 enum: ["Field Worker", "Coordinator", "Fundraiser", "Medical Staff"]
 },
 hoursCommitted: {
 bsonType: "double",
 minimum: 1.0
 }
 }
 }
 },
 validationLevel: "moderate"
})
```

Made with GAMMA

Experiment 5: Viewing Validation Rules

Verify Your Rules

Use `db.getCollectionInfos()` to inspect validation rules applied to any collection. This helps you verify that your rules are correctly implemented.

Query Syntax:

```
db.getCollectionInfos(
 { name: "collection_name" }
)
```

Made with GAMMA

Experiment 5: Complete Query

```
db.getCollectionInfos({ name: "donations" })
db.getCollectionInfos({ name: "volunteers" })
```

Collection Information

Returns metadata including validator rules, creation options, and indexes

Validation Verification

Check if your enum, pattern, and range validations appear in the output

Troubleshooting

If rules aren't showing, verify the collection name and ensure `collMod` executed successfully

Made with GAMMA